

Snippet 1

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$
- **Justify:** The loop checks each element of the array once, taking linear time, while only a few variables (`max_val`, `i`) are used, requiring constant extra space.

Snippet 2

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(1)$
- **Justify:** The two nested loops compare every possible pair of elements, resulting in quadratic time, while only loop variables are used, requiring constant extra space.

Snippet 12

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(n)$
- **Justify:** The function makes one recursive call for each value from `n` down to `1`, giving linear time, and the recursion stack can contain up to `n` calls, requiring linear space.

Snippet 13

- **Time Complexity:** $O(2^n)$
- **Space Complexity:** $O(n)$
- **Justify:** Each call recursively creates two more calls (`n-1` and `n-2`), producing exponential time, while the maximum recursion depth is `n`, requiring linear stack space.